Лабораторная работа № 8.1
Разработка программ с поддержкой технологии CUDA с использованием компилятора командной строки

Цель работы: познакомиться с особенностями разработки программ с использованием технологии CUDA.

Для разработки программ с использованием технологии CUDA необходима установка инструментария из CUDA SDK, CUDA Driver и CUDA Toolkit, в состав которого входит компилятор командной строки `nvcc`. Инструментарий свободно доступен по адресу https://developer.nvidia.com/cuda-downloads/. Программа с поддержкой технологии CUDA разбивается на `.cu`-файлы, компилируемые с использованием `nvcc`, и обычные файлы программы (в данной работе `.cpp`), компилируемые и линкуемые обычным компилятором (в данной работе `cl.exe`, входящим в состав Microsoft Visual Studio).

В состав `.cu`-файлов входят специфичные для технологии CUDA элементы: функции CUDA-ядер (англ. kernel), выполняемые потоковыми мультипроцессорами (англ. Streaming Multiprocessor, SM) видеокарты, и функции, осуществляющие вызов CUDA-ядер. Пример CUDA-ядра, осуществляющего поэлементное умножение векторов:

```
__global__ void VecMulKernel(float *a, float *b, float *c)
{
    // Определение индекса потока
    int i = threadIdx.x + blockIdx.x*blockDim.x;

    // Обработка соответствующей порции данных
    c[i] = a[i] * b[i];
}
```

Пример вызова CUDA-ядра:

```
VecMulKernel<<<blocks, threads>>>(a, b, c);
```

При компиляции `.cu`-файлов может потребоваться подключение ряда заголовочных файлов:

```
#include <cuda.h>
#include <cuda_runtime.h>
//...
```

располагающихся в папке

```
C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v5.0\include\
```

(для Windows XP x86).

`.cpp`-файлы содержат весь остальной код (например, функцию `main()`), включающий вызовы функций из `.cu`-файлов.

Компиляция простейшего проекта с использованием компилятора `nvcc` производится из командной строки следующим образом:

```
nvcc file1.cu file2.cu … fileN.cu file1.cpp file2.cpp … fileN.cpp -o program_name.exe
```

причем для корректной работы пути к файлам `nvcc.exe` и `cl.exe` должны содержаться в переменной окружения `PATH` (при необходимости их нужно добавить в список вручную). Если компиляция завершена успешно, то в результате будет сформирован исполняемый файл с указанным именем (`program_name.exe` в данном примере).

Общая стратегия обработки информации на видеокарте сводится к трем действиям:

1. Передача исходных данных из оперативной памяти в глобальную память видеокарты.
2. Запуск CUDA-ядра.
3. Передача результирующих данных из глобальной памяти видеокарты и оперативную память.

Управление динамической памятью на видеокарте происходит с использованием функций `cudaMalloc()` и `cudaFree()`, функциональность которых аналогична соответствующим функциям стандартной библиотеки для работы с динамической памятью.

Для копирования данных между оперативной памятью и памятью видеокарты применяется функция `cudaMemcpy(pDst, pSrc, SizeInBytes, FromTo)`, в параметрах которой передаются адреса областей памяти источника и приемника, размер копируемой области данных в байтах и константа, определяющая направление копирования:

```
cudaMemcpyHostToHost     // RAM (Host) := RAM (Host)
cudaMemcpyHostToDevice   // GPU (Device) := RAM (Host)
cudaMemcpyDeviceToHost   // RAM (Host) := GPU (Device)
cudaMemcpyDeviceToDevice // GPU (Device) := GPU (Device)
```

При работе с динамической памятью необходимо помнить о том, что указатели на область памяти видеокарты не действительны в основной программе, и, наоборот, указатели на область оперативной памяти не действительны в коде CUDA-ядра.

При вызове функции-ядра в параметрах `blocks` и `threads` указывается конфигурация запуска ядра (число потоков в блоке и число блоков в сетке). Например:

```
// Конфигурация запуска ядра
dim3 threads = dim3(512, 1);        // 512 потоков в блоке
dim3 blocks = dim3(n/threads.x, 1); // n/512 блоков в сетке

// Вызов ядра
MulKernel<<<blocks, threads>>>(a, b, c);
```

В функции-ядре при помощи обращения к предопределенным переменным `threadIdx` и `blockIdx` можно определить «координаты» потока в блоке и блоке в сетке.

Задание. Создать .cu-файл, поместив в него ссылки на заголовочные файлы `cuda.h` и `cuda_runtime.h`, код ядра для поэлементного умножения векторов (приведен выше) и следующий код с вызовом ядра:

```
// a, b - указатели на исходные массивы
// c - указатель на результирующий массив
// n - размер массивов (число элементов)
void vec_mul_cuda(float *a, float *b, float *c, int n)
{
    int SizeInBytes = n * sizeof(float);

    // Указатели на массивы в видеопамяти
    float *a_gpu = NULL;
    float *b_gpu = NULL;
    float *c_gpu = NULL;
```

```
    // Выделение памяти под массивы на GPU
    cudaMalloc( (void **)&a_gpu, SizeInBytes );
    cudaMalloc( (void **)&b_gpu, SizeInBytes );
    cudaMalloc( (void **)&c_gpu, SizeInBytes );

    // Копирование исходных данных из CPU на GPU
    cudaMemcpy(a_gpu, a, SizeInBytes, cudaMemcpyHostToDevice); // a_gpu = a
    cudaMemcpy(b_gpu, b, SizeInBytes, cudaMemcpyHostToDevice); // b_gpu = b

    // Задание конфигурации запуска ядра
    dim3 threads = dim3(512, 1);        // 512 потоков в блоке
    dim3 blocks = dim3(n/threads.x, 1); // n/512 блоков в сетке

    // Запуск ядра (покомпонентное умножение векторов c = a * b)
    VecMulKernel<<<blocks, threads>>>(a_gpu, b_gpu, c_gpu);

    // Копирование результата из GPU в CPU
    cudaMemcpy(c, c_gpu, SizeInBytes, cudaMemcpyDeviceToHost); // c = c_gpu

    // Освобождение памяти GPU
    cudaFree(a_gpu);
    cudaFree(b_gpu);
    cudaFree(c_gpu);
}
```

Создать `.cpp`-файл, поместив в него код для вызова разработанных подпрограмм:

```
#include <iostream>
#include <cuda.h>
#include <cuda_runtime.h>

using namespace std;

// Описание внешней функции, располагающейся в .cu-файле
void vec_mul_cuda(float *a, float *b, float *c, int n);

// Размер вектора (должен быть кратен 512)
const int N = 1024;

// Вектора
float a[N], b[N], c[N];

void main()
{
    // Заполнение векторов исходными данными
    for (int i=0; i<N; i++)
    {
        a[i] = i;
        b[i] = i;
        c[i] = 0;
    }

    // Покомпонентное умножение векторов на GPU
    vec_mul_cuda(a, b, c, 1024);

    // Вывод первых 20 значений результата
    for (int i=0; i<20; i++)
      cout << c[i] << " ";

    getchar();
    return;
}
```

Откомпилировать проект с использованием компилятора командной строки:

```
nvcc имя_cu_файла имя_cpp_файла -o main.exe
```

Убедиться в работоспособности разработанной программы:

Модифицировать программу следующим образом:

- разработать CUDA-ядро для поэлементного сложения векторов;
- сформировать вектора из случайных исходных данных;
- найти сумму векторов с использованием разработанного CUDA-ядра и с использованием подпрограммы на CPU, сравнить полученные результаты, сделать вывод о корректности расчетов на GPU.

Содержание отчета.

1. Титульный лист
2. Цель работы
3. Задание
4. Листинг программы
5. Скриншот с результатами работы программы
6. Выводы

Лабораторная работа № 8.2
Определение параметров видеокарты с поддержкой технологии CUDA в среде Microsoft Visual Studio

Цель работы: познакомиться с особенностями интеграции инструментария CUDA в среде разработки Microsoft Visual Studio, научиться определять основные параметры видеокарт, влияющие на скорость вычислений.

Настройка среды Microsoft Visual Studio

Инструментарий CUDA предоставляет возможность интеграции в состав популярной среды разработки Microsoft Visual Studio, что избавляет разработчика от необходимости напрямую обращаться к компилятору `nvcc` с использованием командной строки или разрабатывать `make`-файлы. Для создания проекта с поддержкой CUDA необходимо (подразумевается, что инструментарий CUDA установлен на машине):

1. Создать новый проект.
2. Добавить к нему необходимые `.cu`-файлы.
3. Для каждого из них указать инструмент, с помощью которого производится компиляция (правый клик по файлу в дереве проекта → Properties → General → Tool → в выпадающем списке выбрать «CUDA Runtime API»):

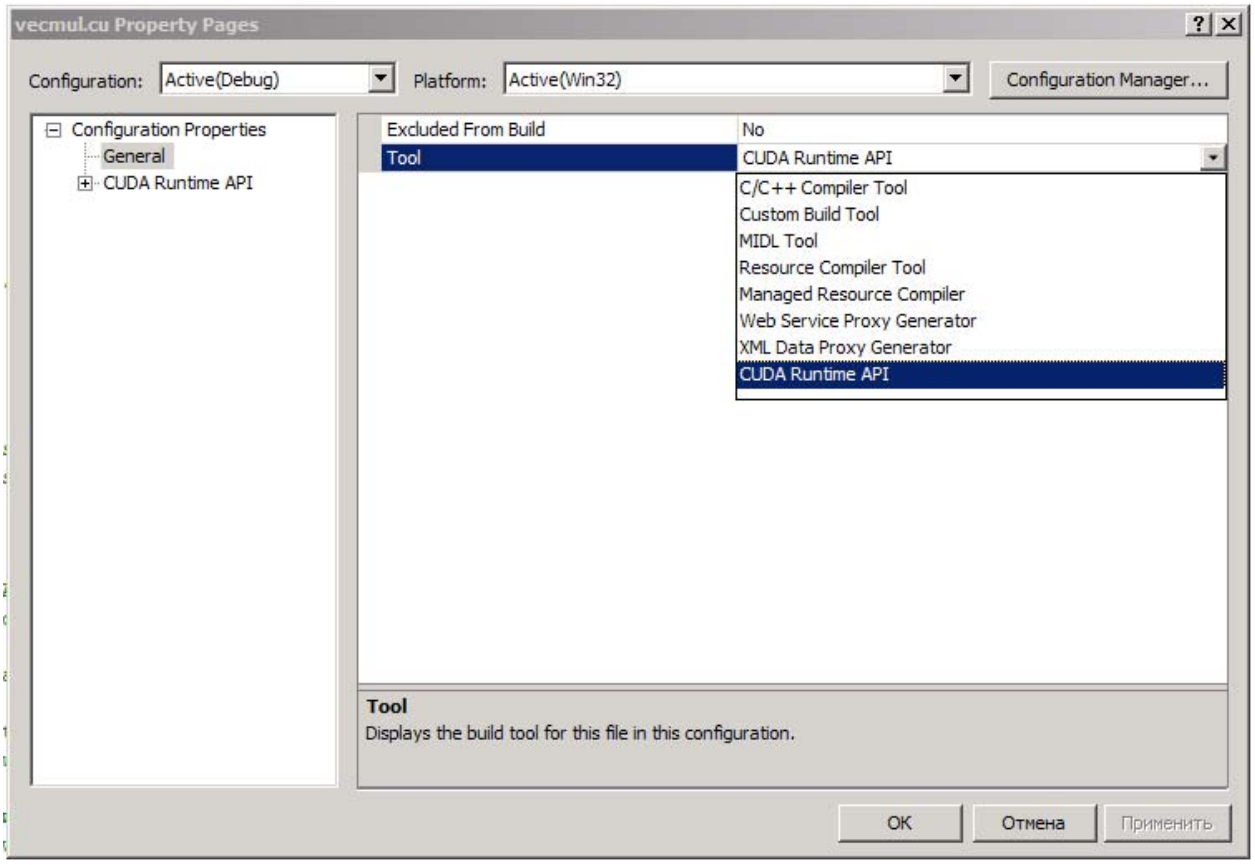


Рис. 1. Выбор типа компиляции для `.cu`-файлов

4. Указать путь к необходимым статически подключаемым библиотекам (выбрать Главное меню → Project → Properties → Configuration properties → Linker → Input → Additional dependencies, указать значение

```
"C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v5.0\lib\Win32\cudart.lib"
```

(в кавычках, путь показан на примере Windows XP x86!)

Другой вариант – указать в коде программы ссылку на библиотеку:

```
#pragma comment(lib,"cudart.lib")
```

5. Указать путь к расположению инструментария CUDA (выбрать Главное меню → Project → Properties → Configuration properties → Linker → Additional library directories, указать значение `$(CUDA_LIB_PATH)`).
6. Указать путь к расположению заголовочных файлов CUDA (выбрать Главное меню → Project → Properties → Configuration properties → C/C++ → Additional include directories, указать значение

```
"C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v5.0\include\"
```

(в кавычках, путь показан на примере Windows XP x86!)

После выполнения указанных действий среда разработки Microsoft Visual Studio будет самостоятельно вызывать компилятор `nvcc` для `.cu`-файлов, осуществлять компиляцию и сборку проекта.

Определение параметров видеокарты

Для определения числа установленных в системе видеокарт с поддержкой технологии CUDA используется функция `cudaGetDeviceCount()`:

```
int device_count;
cudaGetDeviceCount(&device_count);
```

Для определения свойств видеокарты используется функция `cudaGetDeviceProperties()`, принимающая в качестве параметра номер видеокарты и возвращающая в структуре `cudaDeviceProp` интересующие значения:

```
cudaDeviceProp dp;
cudaGetDeviceProperties(&dp, 0);  // Определение параметров GPU с номером 0
```

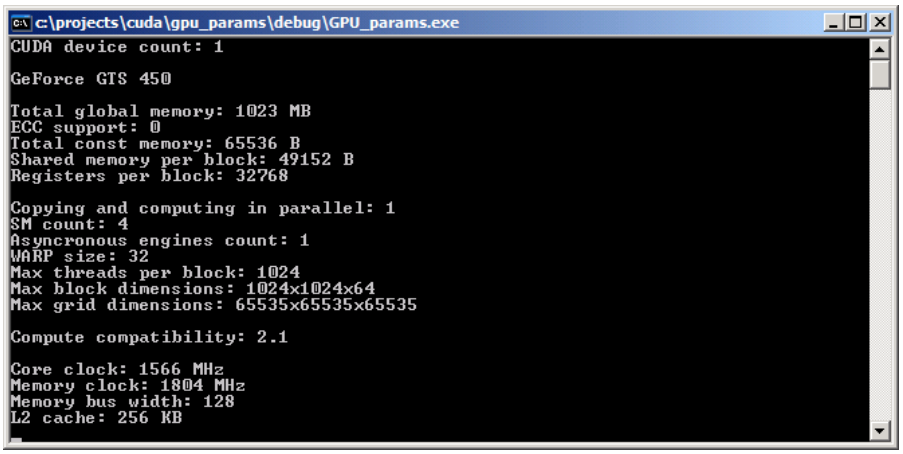
Например, для определения наименований установленных в системе видеокарт с поддержкой CUDA и их вычислительных возможностей можно использовать следующий код:

```
int device_count;
cudaDeviceProp dp;

cudaGetDeviceCount(&device_count);
cout << "CUDA device count: " << device_count << "\n";

for (int i=0; i<device_count; i++)
{
        cudaGetDeviceProperties(&dp, i);
```

```
        cout << i << ": " << dp.name << " with CUDA compute compatibility " <<
            dp.major << "." << dp.minor << "\n";
}
```

Задание.

1. Создать в среде Microsoft Visual Studio проект, состоящий из пары файлов (`.cpp` и `.cu`) из предыдущей работы. Установить в среде необходимые настройки. Убедиться в том, что проект успешно собирается и запускается.
2. Определить число видеокарт с поддержкой технологии CUDA и следующие параметры видеокарты:
    - наименование;
    - общий объем графической памяти;
    - объем памяти констант;
    - объем разделяемой памяти в пределах блока;
    - число регистров в пределах блока;
    - размер WARP'а;
    - максимально допустимое число потоков в блоке;
    - версию вычислительных возможностей;
    - число потоковых мультипроцессоров;
    - тактовую частоту ядра;
    - частоту памяти видеокарты;
    - объем кэша второго уровня;
    - ширину шины памяти видеокарты;
    - максимальную размерность при конфигурации потоков в блоке и блоков в сетке.



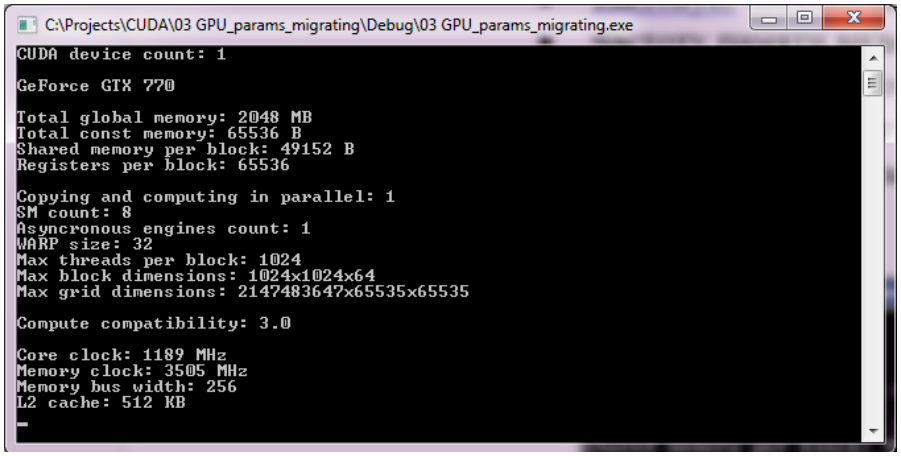


Рис. 2. Основные параметры видеокарты, влияющие на производительность вычислений

Содержание отчета.

1. Титульный лист
2. Цель работы
3. Задание
4. Листинг программы
5. Скриншоты с результатами работы программы
6. Выводы

Лабораторная работа № 8.3
Измерение пропускной способности памяти видеокарт с поддержкой технологии CUDA

Цель работы: научиться измерять пропускную способность памяти GPU при различных типах и направлениях передачи данных.

Работа с памятью является одним из краеугольных камней при разработке эффективных программ с использованием технологии CUDA. Ее правильное использование зачастую позволяет увеличить скорость обработки данных до 10 и более раз.

Видеокарта не имеет прямого доступа к оперативной памяти компьютера (host), поэтому все обрабатываемые данные должны быть загружены в динамическую память GPU (device). В данной работе используется глобальная память, для обмена данными с ней используется функция

```
cudaError_t cudaMemcpy(void *dst, void *src, size_t count, cudaMemcpyKind kind);
```

которая осуществляет копирование `count` байт из источника `src` в приемник `dst`. Направление копирования задается параметром `kind` и может принимать следующие значения:

```
cudaMemcpyHostToHost     /* Host   -> Host */
cudaMemcpyHostToDevice   /* Host   -> Device */
cudaMemcpyDeviceToHost   /* Device -> Host */
cudaMemcpyDeviceToDevice /* Device -> Device */
```

В первом случае копирование производится в пределах оперативной памяти (функция работает как `memcpy()`), во втором – из оперативной памяти в глобальную память GPU, в третьем – из глобальной памяти GPU в оперативную память, и в четвертом – в пределах глобальной памяти GPU.

Для выделения области памяти в глобальной памяти GPU используется функция

```
cudaError_t cudaMalloc(void **devPtr, size_t size);
```

возвращающая в первом параметре указатель на выделенную область памяти размером `size` байт. Для выделения области динамической памяти в оперативной памяти можно использовать функции

```
void * malloc(size_t size);
```

и

```
cudaError_t cudaMallocHost(void **ptr, size_t size);
```

Функция `malloc()` входит в стандартную библиотеку, она выделяет блок динамической памяти размера `size` и возвращает указатель на его начало. Функция `cudaMallocHost()` отличается от нее тем, что помечает выделяемые страницы виртуальной памяти так, что они гарантированно находятся в оперативной памяти и не могут быть вытеснены из нее в файл подкачки (т.н. page locking). Это упрощает процедуру наблюдения за данной памятью со стороны драйвера видеокарты и увеличивает скорость

обмена. Однако пользоваться данной функцией необходимо с осторожностью, т.к. при выделении большого объема данных производительность системы в целом может существенно ухудшиться, могут появиться системные ошибки из-за невозможности выделения заданного объема памяти в других приложениях и т.п.

Копирование в рамках глобальной памяти GPU осуществляется асинхронно: функция `cudaMemcpy()` возвращает управление раньше, чем завершается копирование. Чтобы учесть это, можно добавить в код вызов функции

```
cudaError_t cudaThreadSynchronize();
```

которая ожидает завершения текущей асинхронной операции и лишь затем возвращает управление. Фактически таким образом происходит синхронизация потока CPU с GPU.

Задание.

1. Выделить блоки памяти одинакового размера в оперативной памяти и глобальной памяти видеокарты. Скопировать содержимое блоков между:
   - двумя буферами в оперативной памяти;
   - между оперативной памятью и глобальной памятью видеокарты в направлении CPU→GPU и GPU→CPU с использованием обычной и page-locked памяти (должно быть 4 различных варианта копирования);
   - между двумя буферами в глобальной памяти видеокарты.
2. Убедиться в том, что копирование происходит корректно.
3. Измерить время копирования и, зная размер копируемого блока, определить пропускную способность (в ГБ/с) для каждого типа копирования. Сделать выводы о скорости копирования в различных режимах.
4. В отчет включить краткое описание типа и параметров видеокарты и процессора.

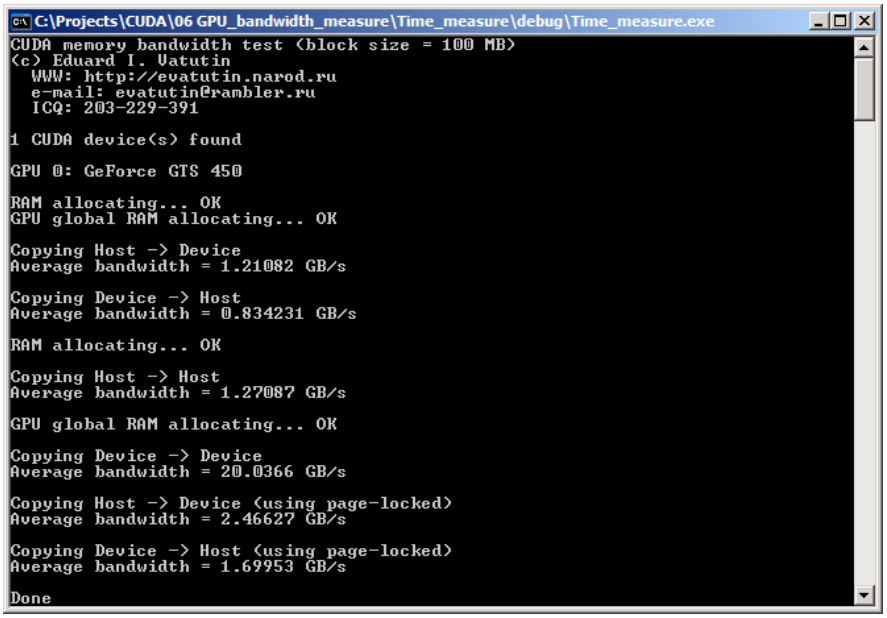


Рис. Результаты измерения пропускной способности

Содержание отчета.

1. Титульный лист
2. Цель работы
3. Задание
4. Листинг программы

5. Результаты измерения пропускной способности
6. Выводы

Лабораторная работа № 8.4
Оптимизация работы с глобальной и разделяемой памятью для видеокарт с поддержкой технологии CUDA

Цель работы: научиться оптимизировать работу с глобальной памятью путем использования разделяемой памяти на примере задачи умножения квадратных матриц.

Работа с памятью является одним из краеугольных камней при разработке эффективных программ с использованием технологии CUDA. Существует ряд задач, требующих интенсивных обменов с памятью (одной из них является рассматриваемая в данной работе задача умножения матриц). Для них реальная производительность видеокарты лимитируется не пиковой производительностью вычислительных блоков, а пропускной способностью памяти, поэтому повышение скорости обработки возможно путем ряда оптимизаций работы с ней.

Глобальная память (англ. global memory) современных видеокарт имеет объем порядка нескольких гигабайт, однако характеризуется достаточно большой задержкой доступа (несколько сотен тактов), обращения к ней разрешены для всех потоков без исключения. При обработке информации можно использовать разделяемую память (англ. shared memory), время обращения к которой составляет единицы тактов и обычно несущественно по сравнению со временем вычислений. Однако разделяемая память характеризуется малым объемом (48 КБ) и доступна только для потоков одного блока. Если обработку исходного большого блока данных можно разбить на малые порции, не превышающие объема разделяемой памяти, то скорость обработки может быть существенно увеличена.

Для размещения переменной в глобальной памяти используется модификатор доступа `__device__`, либо функция `cudaMalloc()`, для размещения в разделяемой памяти – `__shared__`. Пример:

```
__shared__ float AA[S][S]; // Массив в разделяемой памяти
__device__ float A[N][N];  // Массив в глобальной памяти
```

Для копирования данных между глобальной и разделяемой памятью не требуется дополнительных действий со стороны программиста:

```
AA[i][j] = A[i0+i][j0+j];
```

При загрузке временных данных в буфер, расположенный в разделяемой памяти, конфигурация запуска CUDA-ядра обычно выбирается такой, чтобы каждый поток блока копировал одно (реже – несколько) значений. Пример копирования части одномерного массива в разделяемую память из глобальной приведен ниже:

```
const int N = 1024;          // Размер массива
const int S = 512;           // Размер порции данных в разделяемой памяти

__device__ float a[N];       // Массив в глобальной памяти

__global__ void CopyKernel(float *src)
{
        int x = threadIdx.x;
        int i = blockIdx.x * blockDim.x + threadIdx.x;
```

```
        __shared__ float dst[S];      // Массив в разделяемой памяти

        dst[x] = src[i];

        __syncthreads();

        // Обработка загруженной порции данных...
}


void CopyToShared()
{
        CopyKernel<<<dim3(N/S), dim3(S)>>>(a);
}
```

В приведенном примере исходный массив a размером $N = 1024$ элемента, расположенный в глобальной памяти, копируется в разделяемую память порциями по $S = 512$ элементов. При этом необходимо тщательно следить за вычислением индексов копируемых элементов. После завершения копирования необходима обязательная барьерная синхронизация потоков с использованием функции

```
__syncthreads();
```

чтобы убедиться в том, что перед началом обработки скопирована вся порция данных, а не только 32 элемента, скопированных потоками текущего WARP'а.

Если обращения в глобальную память из различных потоков идут по смежным адресам, они, при соблюдении ряда дополнительных условий, могут быть сгруппированы (англ. coalescing), что существенно (до 16 раз) повышает пропускную способность памяти.

Алгоритм умножения квадратных матриц размера $N \times N$ укрупненно можно представить в виде следующей последовательности шагов:

1. Копирование исходных матриц из оперативной памяти в глобальную память видеокарты.
2. Выполнение CUDA-ядра, реализующего умножение.
3. Копирование результирующей матрицы из глобальной памяти видеокарты в оперативную память.

В простейшем случае при выполнении классического умножения «в лоб» без использования разделяемой памяти каждый поток реализует вычисление одного элемента результирующей матрицы $C = AB$ по формуле

$$c_{ij} = \sum_{k=1}^{N} a_{ik} b_{kj}.$$

Значения индексов i и j определяются исходя из конфигурации запуска (число потоков в блоке, число потоков в сетке). Рекомендуемая конфигурация запуска:

```
MatrMulKernel<<<dim3(N/S, N/S), dim3(S, S)>>>();
```

При подобном умножении в глобальную память производится $2N^3$ обращений на чтение. Снизить число обращений к глобальной памяти можно при помощи буферизации j-й строки матрицы A или столбца матрицы B во временной переменной t, хранящейся в разделяемой памяти видеокарты. При этом умножение может быть реализовано по формулам

$$c_{ij} = \sum_{k=1}^{N} a_{ik} t_k$$

или

$$c_{ij} = \sum_{k=1}^{N} t_k b_{kj}$$

в зависимости от выбранного способа. Конфигурацию запуска потоков в блоке рекомендуется выбрать такой, чтобы в каждом потоке производилось вычисление одного элемента результирующей матрицы *C*, а сами элементы были сгруппированы в строку или столбец в зависимости от способа:

```
MatrMulBuffKernel<<<dim3(N), dim3(N)>>>();
```

При этом следует помнить, что для современных видеокарт максимальное число потоков в блоке ограничено значением 1024, что может быть ограничением на размер обрабатываемых матриц и должно быть учтено в коде CUDA-ядра.

При подобной буферизации производится $N^3 + N^2$ обращений к глобальной памяти на чтение и N^3 обращений к разделяемой памяти. Дополнительно снизить число обращений к глобальной памяти можно с использованием блочного подхода, при котором результат рассчитывается блоками размером $S \times S$ элементов. Каждый подобный блок C'_{xy} результирующей матрицы $C = \begin{Vmatrix} C'_{11} & C'_{12} & \dots \\ C'_{21} & C'_{22} & \dots \\ \dots & \dots & \dots \end{Vmatrix}$ обрабатывается группой потоков, входящих в состав блока, что соответствует конфигурации запуска

```
MatrMulBlockKernel<<<dim3(N/S, N/S), dim3(S, S)>>>();
```

При этом потоки блока поочередно загружают в разделяемую память блоки A'_{xz} и B'_{zy} матриц *A* и *B* (каждый блок также имеет размер $S \times S$ элементов). По окончании загрузки подматриц каждый поток блока считает свой элемент блока C'_{xy}

$$c'_{ij} = c_{x+i,\, y+j} = \sum_{k=1}^{S} a'_{ik} b'_{kj} = \sum_{k=1}^{S} a_{x+i,\, k} b_{k,\, y+j} \ ,$$

обращаясь к закэшированным элементам блоков A'_{xz} и B'_{zy}. Затем производится загрузка следующих блоков для $z = z + 1$. По завершении обработки всех подматриц A'_{xz} и B'_{zy}, $z = \overline{1, \frac{N}{S}}$ результирующие значения элементов c'_{ij} записываются в глобальную память (каждый поток блока считает одно подобное значение).

Данный подход требует $2N^2$ обращений к глобальной памяти на чтение и $2N^3$ обращений к разделяемой памяти.

При подобной обработке в зависимости от способа определения индексов *i*, *j*, *x* и *y* в коде потоков (зависят от координат потока в блоке и блока в сетке) могут быть нарушены условия coalescing-обращений к глобальной памяти, на что необходимо обратить внимание при разработке кода CUDA-ядра. При необходимости можно поменять индексы местами, контролируя корректность получаемого результата и общее время выполнения.

Для использования потенциала параллелизма на уровне команд (англ. Instruction Level Parallelism, ILP) код циклов умножения матриц можно раскрутить на некоторое количество итераций (2, 4, 8, …), контролируя корректность результата и время выполнения умножения.

Задание.

1. Разработать CUDA-ядра для вариантов классического умножения матриц (без оптимизации), с кэшированием строки, столбца и для блочного умножения.
2. Разработать функции, реализующие копирование исходных матриц в глобальную память видеокарты, вызов соответствующего CUDA-ядра и обратное копирование результата.
3. Реализовать CPU-аналог умножения матриц (можно взять из предыдущих работ).
4. Организовать измерение времени выполнения умножения матриц для всех реализованных способов.
5. Реализовать подпрограмму, проверяющую корректность выполнения умножения путем сопоставления двух результирующих матриц. Убедиться в совпадении результатов для всех способов реализации умножения в независимости от выполненных оптимизаций. С использованием функции `cudaGetLastError()` убедиться в отсутствии ошибок.
6. Измерить время выполнения разработанных CUDA-ядер, выигрыш по сравнению с реализацией на процессоре и реальную производительность при умножении матриц различного размера (для простоты размер N взять равным степени двойки). При подсчете реальной производительности считать, что в наиболее вложенном цикле выполняется 2 операции с плавающей точкой (умножение и сложение), общее число операций с плавающей точкой составляет $2N^3$, а реальная производительность $P = \frac{2N^3}{t}$, где t – время выполнения действия. Построить графики зависимости:
   - времени выполнения умножения и реальной производительности от размера блока для матриц различного размера для каждого из алгоритмов;
   - времени выполнения и реальной производительности от степени раскрутки цикла для выбранных значений N и S для каждого из алгоритмов;
   - времени выполнения и реальной производительности для алгоритмов с кэшированием строки и столбца в зависимости от конфигурации запуска (числа потоков в блоке).
7. Сформулировать выводы и рекомендации о наилучшем способе умножения матриц с использованием GPU и параметрах запуска его CUDA-ядра.

Содержание отчета.

1. Титульный лист
2. Цель работы
3. Задание
4. Листинги программы
5. Результаты измерения времени выполнения, производительности и выигрыша в скорости. Соответствующий графический материал.
6. Выводы